

Presentación Didáctica: Introducción a C# 14, EF Core 10 y Razor Pages

Esta presentación está diseñada como material didáctico para introducir los conceptos fundamentales del desarrollo web moderno con el stack de .NET 10 a partir del código de la aplicación de **Gestión de Tareas**.

Índice de la Presentación

1. **Módulo 1: Fundamentos de C# 14 y .NET 10**
 - Constructores Primarios (Primary Constructors)
 - Inmutabilidad con `record` y DTOs
 - Tipos de Referencia Nulos (?) y Propiedad `required`
 2. **Módulo 2: Entity Framework Core 10 & SQLite**
 - Mapeo de Entidades y `DbContext`
 - Rendimiento Extremo: `AsNoTracking()`
 - Operaciones Masivas Directas: `ExecuteUpdateAsync` y `ExecuteDeleteAsync`
 - Migraciones y SQLite
 3. **Módulo 3: ASP.NET Core Razor Pages**
 - El Modelo Page-Centric y `PageModel`
 - ¿Por qué usamos Métodos Asíncronos (`async/await`)? Escalabilidad y Thread Pool
 - Inyección de Dependencias
 - El Patrón PRG (Post-Redirect-Get) y `TempData`
 - Vistas Limpias con Tag Helpers (`asp-for`, `asp-page`)
 4. **Módulo 4: Arquitectura y Buenas Prácticas**
 - Separación de Responsabilidades (SRP)
 - Cancelación Asíncrona con `CancellationToken`
-

Módulo 1: Fundamentos de C# 14 y .NET 10

1.1 Constructores Primarios (Primary Constructors)

En C# 14 / .NET 10, ya no es necesario escribir campos privados explícitos ni constructores repetitivos para la Inyección de Dependencias.

Ejemplo en el proyecto (`TareasService.cs`):

```
// Los parámetros de la clase actúan directamente como dependencias
// disponibles en toda la clase
public class TareasService(AppDbContext db, ILogger<TareasService> logger)
: ITareasService
{
    // db y logger están disponibles sin necesidad de declararlos
    // previamente como campos 'private readonly'
}
```

1.2 Inmutabilidad y Modelado de Datos con **record**

Los **record** son tipos por referencia que proporcionan igualdad estructural e inmutabilidad por defecto, ideales para DTOs (Data Transfer Objects) y comandos.

Ejemplo en el proyecto (Tarea.cs):

```
// DTOs e intenciones de comando representados de forma concisa e inmutable
public record TareaDto(Guid Id, string Descripcion, bool EstaAcabada,
    DateTime? Fecha);
public record CreateTareaCommand(string Descripcion, DateTime? Fecha);
```

1.3 Seguridad Nula y Propiedad **required**

C# evita errores comunes como **NullReferenceException** indicando explícitamente qué campos son obligatorios u opcionales.

- **required string Descripcion**: Garantiza que el objeto no puede instanciarse sin asignar una descripción.
- **DateTime? Fecha**: Indica mediante **?** que la fecha es un valor opcional (puede ser **null**).

Módulo 2: Entity Framework Core 10 & SQLite

Entity Framework Core (EF Core) es el ORM (Object-Relational Mapper) oficial de .NET que traduce operaciones en código C# a consultas SQL nativas.

2.1 El Contexto de Base de Datos (**AppDbContext**)

Define las tablas que existirán en la base de datos y sus reglas de mapeo.

Ejemplo (AppDbContext.cs):

```
public class AppDbContext(DbContextOptions<AppDbContext> options) :
    DbContext(options)
{
    public DbSet<Tarea> Tareas => Set<Tarea>();
}
```

2.2 Consultas de Lectura Optimizadas (**AsNoTracking**)

Cuando leemos datos solo para mostrarlos en pantalla y no los vamos a modificar inmediatamente, desactivamos el rastreador de cambios (*Change Tracker*) para ahorrar memoria y acelerar la consulta.

Ejemplo (TareasService.cs):

```
public async Task<IReadOnlyList<TareaDto>>
GetTodasLasTareasAsync(CancellationToken ct = default)
{
    return await db.Tareas
        .AsNoTracking() // Desactiva el seguimiento -> Máximo Rendimiento
        .OrderByDescending(t => t.Fecha)
        .Select(t => new TareaDto(t.Id, t.Descripcion, t.EstaAcabada,
t.Fecha))
        .ToListAsync(ct);
}
```

2.3 Operaciones Masivas Eficientes (`ExecuteUpdateAsync` / `ExecuteDeleteAsync`)

En lugar de traer un registro a la memoria, modificarlo y guardarlo (lo que requiere 2 viajes a la base de datos), ejecutamos la orden directamente en el motor SQLite en 1 sola sentencia SQL.

Ejemplo (`TareasService.cs`):

```
// Cambia el estado directamente en SQL sin cargar la entidad en memoria
int updatedRows = await db.Tareas
    .Where(t => t.Id == id)
    .ExecuteUpdateAsync(setters => setters.SetProperty(t => t.EstaAcabada,
t => !t.EstaAcabada), ct);
```

🌐 Módulo 3: ASP.NET Core Razor Pages

Razor Pages combina el código de interfaz (HTML/Razor `.cshtml`) y la lógica de backend (`.cshtml.cs`).

3.1 El Modelo Page-Centric

Razor Pages estructura el código web asociando cada vista `.cshtml` a su clase de soporte `.cshtml.cs`.

3.2 ¿Por qué usamos Métodos Asíncronos (`async` / `await`)?

En aplicaciones web de alto rendimiento, **toda operación de Entrada/Salida (I/O)** —como consultar a la base de datos o leer archivos— debe ser **asíncrona** (`async Task`).

- **Programación Sincrónica (Bloqueante):** El hilo del servidor se queda bloqueado esperando a que la base de datos responda. Si llegan muchas peticiones a la vez, se produce un agotamiento del grupo de hilos (*Thread Starvation*) y la web deja de responder.
- **Programación Asíncrona (`async/await`):** Al llegar al operador `await`, el hilo del servidor **se libera inmediatamente** para atender peticiones de otros usuarios mientras la base de datos procesa la consulta. Al finalizar, un hilo disponible retoma la ejecución.

Regla de Oro Didáctica: `async/await` no incrementa necesariamente la velocidad de una sola petición individual, pero **multiplica masivamente la capacidad del servidor para atender a cientos de usuarios simultáneos con los mismos recursos**.

3.3 El Manejador **PageModel** y el Patrón PRG (Post-Redirect-Get)

Para evitar que el usuario reenvíe un formulario por accidente al refrescar la página tras una acción **POST**, se usa el patrón **PRG**:

1. El usuario envía el formulario (**POST**).
2. Se procesa la información y se responde con una redirección (**Redirect**).
3. El navegador realiza una nueva petición de lectura (**GET**).

Ejemplo (**Create.cshtml.cs**):

```
public async Task<IActionResult> OnPostAsync(CancellationToken ct)
{
    if (!ModelState.IsValid) return Page(); // Si hay error, recarga la
    vista

    await tareasService.CreateTareaAsync(new
    CreateTareaCommand(Input.Descripcion, Input.Fecha), ct);

    TempData["Mensaje"] = "Tarea creada exitosamente.";
    return RedirectToPage("./Index"); // Redirección (PRG)
}
```

3.4 Tag Helpers en Vistas Razor

Los Tag Helpers permiten escribir marcado HTML limpio sintácticamente asistido por C#.

Ejemplo (**Create.cshtml**):

```
<!-- asp-for vincula el input directamente con la propiedad C# y genera id,
name y tipo adecuado -->
<input asp-for="Input.Descripcion" class="form-control" />
<span asp-validation-for="Input.Descripcion" class="text-danger"></span>
```

Módulo 4: Arquitectura y Buenas Prácticas

1. Separación de Responsabilidades (SRP):

- Las Razor Pages (**PageModel**) solo manejan peticiones HTTP y la interfaz.
- El servicio **TareasService** contiene la lógica de negocio y llamadas a la base de datos.

2. Propagación de **CancellationToken**:

- Permite cancelar consultas a la base de datos si el usuario cierra el navegador o cancela la petición antes de que termine.

Conclusión para los Alumnos

Esta arquitectura demuestra cómo construir aplicaciones web en .NET que son al mismo tiempo **simples de mantener, sólidamente estructuradas y optimizadas para alto rendimiento** utilizando los estándares modernos de **C# 14, EF Core 10 y Razor Pages**.